

Notes for ECE 46800 - Introduction to Compilers and Translation Engineering

Zeke Ulrich

September 4, 2026

Contents

<i>Course Description</i>	1
<i>Compiler Basics</i>	2
<i>Phases of a Compiler</i>	3
<i>Regular Languages</i>	4
<i>Automata</i>	6
<i>Regex Engines</i>	7
<i>Context-free Grammar</i>	8
<i>Parsing</i>	9

Course Description

The design and construction of compilers and other translators. Topics include compilation goals, organization of a translator, grammars and languages, symbol tables, lexical analysis, syntax analysis (parsing), error handling, intermediate and final code generation, assemblers, interpreters, and an introduction to optimization. Emphasis is on engineering a compiler or interpreter for a small programming language - typically a C or Pascal subset. Projects involved the stepwise implementation (and documentation) of such a system.

Compiler Basics

A *compiler* is commonly understood as a program that converts a high-level language to a low-level language executable by hardware. However, in general a compiler translates one representation of a program to another, and perhaps does some analysis along the way.

All translation is transformation. A Greek-to-English translation of the Odyssey will preserve the meaning while reading as close as possible to the original. Code translation is the same. There are many possible translations of a given program to another language, with different properties and optimizations. Two translations that accomplish the same task may run in different times or use different amounts of memory.

Compilers save poor programmers from writing directly in assembly. The many flavours of high-to-low compilers enable us to write in one language and compile to many different architectures, instead of needing to write the same program in x86 for x86 machines, RISC-V for RISC-V chips, and so on.

Regular Languages

Remember the job of the scanner is to identify the "words" in a program. Variable names, keywords, operators. We need to define what those words are, and we do so with regular expressions. Regular expressions allow us to determine what is valid syntax and what is user-typed gibberish that happens to be typed in an IDE.

A language is a set of strings, or words, composed of symbols from a finite alphabet. To get from a generic language to a regular language, we need to define a regular set.

- The empty set $\emptyset$ is a regular set.
- A singleton $S = \{a\}$ is a regular set.
- A union of two regular sets is a regular set: $S_1 = \{a\}, S_2 = \{b\}, S_3 = S_1 \cup S_2 = \{a, b\}$.
- The concatenation of two regular sets is a regular set: $S_1 = \{a, b\}, S_2 = \{c, d\}, S_3 = S_1 \times S_2 = \{ac, ad, bc, bd\}$.
- The empty string $S = \{\epsilon\}$ is a regular set.

More generally, any finite set of strings is a regular set.

Kleene star is an operator that generates every possible concatenation of a regular set. E.g., $S = \{a, b\}, S^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$.

Now, finally to regular expressions (regexes). A regular language is exactly those languages which are defined by regular expressions. Basic regular expression syntax corresponds to operations on regular sets.

- A string of characters is a regular set: $r_1 = \text{Hello}$
- The choice operator unions together two regular sets: $r_3 = r_1 | r_2$.
- The star operator repeats a regular set 0 or more times: $r_2 = r_1^*$.
- Can use parentheses for operator precedence: $r = (\text{Hello} * (\text{world} | \text{class}))$.

For convenience, regular expression engines provide a lot of syntax that makes it easier to define regular sets (but do not make regular expressions more expressive). See any regex cheat sheet.

A scanner needs to find the words in a language. It checks if a sequence of characters matches some regular expression. It requires a systematic procedure for converting a regular expression into code that recognizes when a string matches a regular expression, and a way this is done is with finite automata, represented with transition tables. Recall an automaton is deterministic if at each step there is only one possible transition to take on a given character. But an automaton can be non-deterministic: at a particular state, there may be more

than one transition to take on a given character. There may also be λ -transitions which consume no characters. To run a non-deterministic automaton, go through, taking every path, and if any path reaches an accept state then the input is accepted.

Automata

We use both non-deterministic and deterministic automata to convert regex to a lexer.

Every regex expression has a non-deterministic finite automaton that matches, and a deterministic finite automaton (DFA) can be algorithmically made from any NFA according to the rules in Figure .

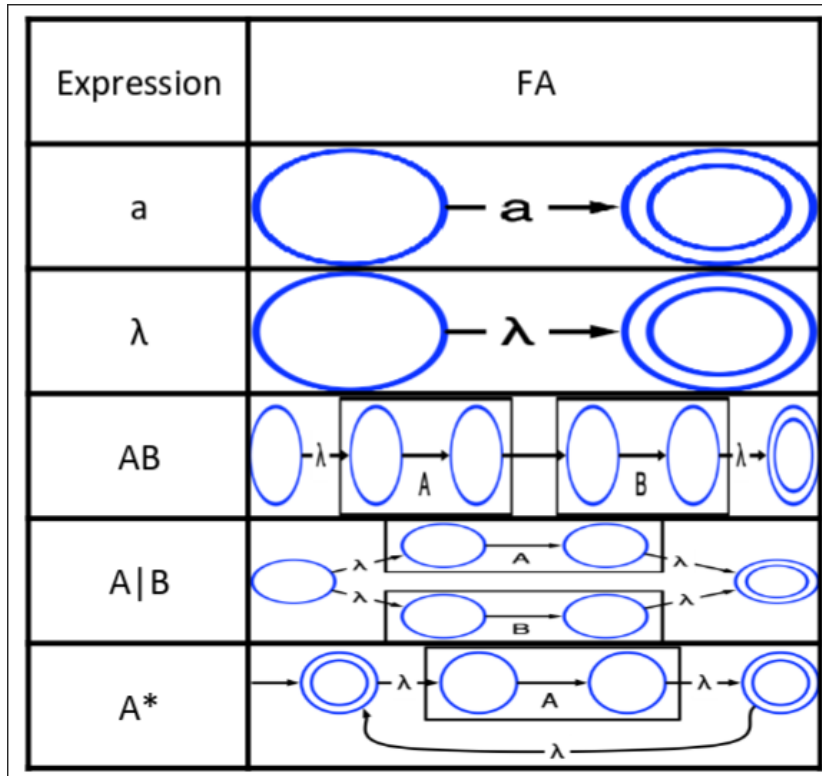


Figure 1: NFA to DFA Conversion Rules

The DFA built from an NFA according to these rules is not necessarily optimized, so it should be minimized using transition partitioning. Two states are equivalent if they have the same accepting/not accepting status and if they have the same transitions for a given input. If state A and B are both accepting and both move to state C on input a , then they are the same state. Keep merging states according to these rules until there are no changes to get the minimized DFA.

Regex Engines

Using a transition table, it is straightforward to write a program to recognize strings in a regular language. If T is a matrix representing the transition table with state rows and character columns, then it might look something like:

```

state = initial_state; //start state of FA
while (true) {
    next_char = getc();
    if (next_char == EOF) break;

    next_state = T[state][next_char];

    if (next_state == ERROR) break;
    state = next_state;
}
if (is_final_state(state))
    //recognized a valid string
else
    handle_error(next_char);

```

Up until now, we have only considered matching an entire string to see if it is in a regular language. If we want to match multiple tokens from a file, we need to look ahead to see if the next character belongs to the current token. If it does, we can continue. If it doesn't, the next character becomes part of the next token. We don't need to care about these details or tie-breaking or anything else when using ANTLR, as it takes care of everything behind the scenes.

After a program is tokenized, we need a parser to determine the structure of the program.

Context-free Grammar

The structure of a program is recursive: If statements nested inside while loops nested inside for loops nested inside if statements nested.... Nesting can be arbitrarily deep. Accounting for this kind of recursive nesting is beyond what regular expressions can do and requires context-free grammar (CFG).

- Grammar $G = (V_t, V_n, S, P)$
- V_t is the set of terminal symbols
- V_n is the set of variables/non-terminal symbols
- S is the start symbol
- P = set of production rules

Programming language syntax is defined with CFGs.

Parsing

Using a grammar to generate a string is straightforward. Parsing solves the opposite problem: is a string part of a language? Or, is there some combination of rewrites that will generate a string, for whatever rewrite are valid under the rules?

A parse tree shows how a string was generated.

- Interior nodes: non-terminals
- Leaf nodes: terminals
- Children of interior nodes: the terminals and non-terminals generated by applying a rule

Parsing is recognizing members in a language specified/defined/generated by a grammar. The parse tree is like a sentence diagram.

In practice, the most common parser structure is top-down, meaning it functions by expanding the parse tree from the root node down. It expands the tree in pre-order, and for each node in the parse tree, figures out what it expands to. Commonly involves mutually-recursive functions or LL(I) parser, a table-based parser that operates similarly to recursive-descent.